
Sign Language Alphabet Recognition

Kyana Marckx (852793982)¹ Bo Alen (852801681)¹

Abstract

This project investigates the feasibility of recognizing American Sign Language (ASL) alphabet letters using MediaPipe hand landmarks and a machine learning classifier. A complete pipeline was developed, including dataset preprocessing, landmark extraction, model training, and real-time prediction. Several models were evaluated, with a Multi-Layer Perceptron (MLP) neural network achieving the highest accuracy after hyperparameter tuning. To assess real-world performance, additional experiments were conducted under varying lighting conditions, camera angles, distances, and letter types. The results show that the model performs reliably under good lighting and favorable viewing angles, but accuracy decreases under poor lighting and for letters with similar hand shapes. These findings demonstrate the potential of lightweight, landmark-based models for sign language recognition, while also revealing the need for greater data diversity and improved robustness.

1. Brief Introduction

Automatic recognition of sign language has become increasingly feasible due to advances in computer vision and real-time hand tracking. Modern landmark-based approaches such as MediaPipe allow systems to estimate hand poses using only a standard webcam, removing the need for specialized hardware.

The goal of this assignment is to build and evaluate a system that recognizes letters of the ASL alphabet. The project covers the full pipeline from data preprocessing and model training to real-time prediction and controlled testing under different conditions. In this context, we address the following research questions:

- **RQ1:** How accurately can a landmark-based model classify ASL letters?
- **RQ2:** How robust is the model to variations in lighting, camera angle, and distance?

2. Related Works

Research on sign language recognition has evolved from hardware-based solutions to modern computer vision approaches. Earlier systems relied on wearable devices such as data gloves or sensor-equipped hardware to capture precise finger positions, enabling reliable recognition but limiting practical usability due to cost and lack of flexibility (Pugeault & Bowden, 2011).

With advances in computer vision, image-based methods became dominant, enabling recognition directly from RGB video streams (Simon et al., 2017). However, these methods typically require large datasets and computationally intensive deep learning architectures (Tran et al., 2015). To address these limitations, recent work has focused on lightweight landmark-based techniques.

Google's MediaPipe framework provides a significant step forward by offering fast and accurate real-time hand landmark detection using only a standard webcam (Google AI, 2023; Lugaresi et al., 2019). Instead of relying on raw pixel data, MediaPipe extracts 21 keypoints per hand, enabling machine learning models to classify gestures more efficiently and with greater robustness to background variation and lighting changes. Landmark-based approaches have been shown to perform well for static ASL letters, especially when combined with classical machine learning models such as SVMs, Random Forests, or MLPs.

Public datasets such as the Roboflow ASL Letters dataset¹ have contributed to the accessibility of research in this domain by providing labeled images for training and evaluation. However, these datasets often suffer from class imbalance or incomplete letter coverage, which can limit generalization in real-world scenarios.

Overall, the current literature supports the effectiveness of real-time landmark-based methods for static sign recognition while highlighting the need for more diverse and balanced datasets to improve robustness.

¹<https://public.roboflow.com/object-detection/american-sign-language-letters/1>

3. Methods

The pipeline consists of five main components: installations, MediaPipe usage, data preprocessing, model training and lastly real-time prediction. The real-time prediction stage also includes a model testing component, which allows controlled experimentation with varying angles, distances, and lighting conditions to evaluate the system's robustness.

Each component plays a crucial role in building a complete pipeline for sign language alphabet recognition. The full implementation, including all notebooks, scripts and pre-processed data, is available in [this GitHub repository](#). It is recommended to read the `README.txt` before starting the project or running the notebooks.

3.1. Installations

For the first step of the implementation, several essential Python packages were installed in the notebook `1-installations.ipynb`.

- **MediaPipe**: used for hand landmark detection and pose estimation.
- **OpenCV**: used for image and webcam processing.
- **Scikit-Learn**: used to train and evaluate multiple machine learning classifiers.
- **NumPy** and **Pandas**: used for numerical computation and structured data manipulation.
- **Matplotlib** and **tqdm**: used for visualization and progress tracking.
- **Joblib**: used for saving and loading trained models and scalers.

Note: ensure that Python version `3.12.xx` is installed, as the Mediapipe package will not install or run correctly with other (newer) versions.

3.2. MediaPipe usage

Google's MediaPipe Hand Landmarker² was used as the core tool to detect and extract 3D hand landmarks. Each detected hand is represented by 21 key points corresponding to joint positions in a three-dimensional space (x, y, z). This component was first tested in `2-mediapipe.ipynb` on one static image of a woman with both hands visible to verify correct detection and visualization of hand skeletons before being applied to the entire dataset. The resulting landmarks will serve as the main input features for the classification stage.

²https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

3.3. Data preprocessing

The preprocessing stage focused on converting the original COCO-format dataset into numerical landmark-based data suitable for training machine learning models. The workflow in `3-preprocessing.ipynb` includes:

1. Mapping image IDs to labels using the COCO annotation files.
2. Extracting hand landmarks from all dataset images using MediaPipe.
3. Structuring the extracted features into training, validation and test subsets.
4. Normalizing the coordinates to achieve scale- and rotation-invariance, reducing sensitivity to hand size or camera position.
5. Standardizing all features using a Scikit-Learn *StandardScaler* to ensure consistent scaling.

The processed datasets and the scaler object were saved in a single NumPy `.npz` file for reproducibility.

3.4. Model training

Several machine learning models were trained inside the `4-model.ipynb`-notebook to classify ASL letters based on the extracted hand landmark data. Four models were implemented and compared: *K-Nearest Neighbors (KNN)*, *Random Forest (RF)*, *Support Vector Machine (SVM)* and a *Multi-Layer Perceptron (MLP)* neural network. After the initial training, a manual grid search was conducted to fine-tune hyperparameters for each model, exploring different combinations to improve performance. The MLP classifier achieved the highest accuracy across both validation and test sets and was therefore selected as the final model. Finally, both the trained model and the scaler were saved as serialized `.pkl` files using Joblib for later use during real-time prediction.

3.5. Real-time prediction

The final component integrates the trained model into a live recognition system in `5-testing.ipynb`. The saved MLP model and scaler are loaded, and MediaPipe is initialized alongside OpenCV to capture frames from a webcam. For each frame, hand landmarks are extracted, preprocessed, and passed to the classifier to predict the corresponding ASL letter.

The output is displayed directly on the video stream, showing both the predicted letter and a confidence score. This step demonstrates the real-world applicability of the system, allowing dynamic hand gesture recognition in real

time using a standard webcam camera. Additionally, a dedicated Python script `6-app.py` was developed based on the testing notebook, enabling users to run the real-time sign language recognition system more easily without navigating through all the individual cells.

3.5.1. MODEL TESTING

To evaluate the performance of the sign language detection model, the second part of the code (starting from "*Fifty frames detection*" in the notebook) was added to enable systematic and repeatable testing. The model was executed in a controlled loop in which a prediction was generated every two seconds. This interval ensured that each prediction was based on a stable hand gesture rather than continuous movement. Between each captured frame, the hand was intentionally closed into a fist to clearly separate consecutive gestures and to prevent misclassifications caused by transitional movements.

The testing script was further modified to automatically stop after 50 iterations, resulting in a fixed number of prediction attempts for each target letter. During each iteration, the program recorded whether the correct target letter was detected and also logged any other letters predicted by the model. This enabled both accuracy calculation and analysis of the model's confusion patterns.

To assess the robustness of the system, multiple environmental and positional conditions were systematically changed during testing. Specifically, the distance between the hand and the camera was changed, the lighting conditions were altered, the camera angles were modified, and five different letters were tested. The following factors were included:

- **Lighting conditions:** good lighting and poor lighting
- **Camera/screen angle:** 90°, 120° and 145°
- **Hand-to-camera distance:** 30 cm, 50 cm and 80 cm
- **Letters tested:** A, C, L, M and O

For each combination of conditions, the model performed 50 prediction iterations. At the end of each test run, the program counted:

1. how many times the target letter was correctly predicted, and
2. which alternative letters were mistakenly detected.

This testing procedure provided a consistent and reproducible evaluation of the model's reliability under varying environmental setups and allowed systematic comparison of recognition performance across different letters and test conditions.

Overall, this five-step methodology provides a complete and reproducible pipeline for ASL Alphabet Recognition.

4. Experimental Results

To evaluate how well the trained MLP model performs in real-world situations, a series of controlled tests were conducted in which lighting conditions, camera angles, hand-to-camera distances, and letters were systematically varied. For every combination, 50 prediction attempts were recorded. The benchmark experiments on the processed ASL dataset had already shown that the tuned MLP achieves the highest average accuracy (0.8510) among all evaluated models. The additional experiments described here focus on how this model behaves when deployed in a realistic webcam setting.

Across the different conditions, the results demonstrate that the model performs best under good lighting, a 90° camera angle, and short distances (30-50 cm). Under these optimal conditions, letter C was detected almost perfectly, with scores ranging from 47/50 to 50/50 correct detections. Increasing the distance to 80 cm caused only a small decrease in performance for this letter (48/50 correct). However, larger camera angles such as 145° caused a substantial drop, with only 23/50 frames correctly classified.

Poor lighting negatively affected recognition, particularly for the letter C at 50 cm distance, where accuracy dropped from 50/50 to 40/50 correct detections and more misclassifications were observed. This confirms that illumination strongly influences the model's ability to correctly extract hand landmarks.

For the other letters, results varied more widely. The letter A was recognized well (42/50 correct), and L showed similarly strong results (38/50 correct). The letter M, however, was more challenging for the model: although 47 frames contained a successful detection of some letter, only 28 were correctly recognized as M, with confusions primarily involving A, E, F, and N. The most difficult letter for the model was O, for which only 14/50 frames were correctly detected; many predictions were incorrectly classified as C, Q, or R. These confusions suggest that certain shapes are more difficult for the model to distinguish, likely due to similarities in hand posture or limited representation in the training dataset.

Overall, the results show that the model behaves consistently and accurately in well-lit conditions and at favorable camera angles. Performance decreases with increasing angle, poor lighting, and for letters that exhibit similar visual characteristics. This highlights the need for improved robustness, particularly through additional training data for challenging gestures and more diverse environmental conditions.

5. Discussion

The experimental results show that the landmark-based classifier performs reliably when environmental conditions are favorable. Good lighting, a stable hand pose, and a 90° camera angle consistently led to high recognition rates, especially for letters with distinctive shapes such as C, A, and L. In contrast, performance dropped noticeably under poor lighting, at wider camera angles, or when gestures involved more subtle finger configurations. Letters like M and O were misclassified frequently, confirming that some hand shapes are inherently more difficult for the model to separate.

These observations correspond well with earlier work in computer vision. Image-based deep learning systems have traditionally achieved strong performance on gesture and sign recognition but heavily depend on large datasets and considerable computational resources (Simon et al., 2017; Tran et al., 2015). Lightweight landmark-based approaches such as MediaPipe aim to reduce this dependency by extracting geometric information directly (Google AI, 2023; Lugaresi et al., 2019). Our findings support this: the classifier works well when reliable landmarks are available but becomes unstable when detection quality is affected by lighting or orientation.

A key limitation of the current system lies in the training dataset. Not all ASL letters were equally represented, and some were missing entirely in certain subsets, which introduced class imbalance. This imbalance was reflected in the model’s predictions: letters with many examples were recognized more accurately, while letters with fewer examples contributed to higher confusion rates. Improving the balance and diversity of the dataset by gathering additional samples or applying targeted data augmentation, would likely strengthen the overall robustness of the system.

Future work could address these limitations by expanding the dataset, improving robustness to lighting and angle variations, and exploring temporal models for dynamic signs. Such improvements would enhance generalization and increase the practical applicability of landmark-based ASL recognition in real-world conditions.

6. Tables and figures

6.1. Tables

Table 1 shows the initial accuracy of each model on the test set, validation set, along with the accuracy across these two splits. Table 2 summarizes the accuracy improvements after the manual grid search. Table 3 presents the MLP classification report with precision, recall, F1-score and class support; the final row shows the weighted averages per metric. Table 4 reports real-world results for five tested letters over 50 frames under varying lighting, angles and distances. The

column *Other letters* lists the incorrect predictions.

Model	Test Acc	Valid Acc	Avg Acc
KNN	0.6885	0.6316	0.6601
RF	0.7869	0.7519	0.7694
SVM	0.8033	0.7594	0.7813
MLP	0.8197	0.8045	0.8121

Table 1. Initial accuracy per model

Model	Original	Tuned	Improvement
KNN	0.6601	0.7311	+0.0711
RF	0.7694	0.7694	+0.0000
SVM	0.7813	0.8015	+0.0202
MLP	0.8121	0.8510	+0.0389

Table 2. Comparison between original and tuned models based on average accuracy

Letter	Precision	Recall	F1	Support
A	1.00	1.00	1.00	1
B	0.75	1.00	0.86	3
C	1.00	0.67	0.80	3
D	1.00	1.00	1.00	1
F	1.00	1.00	1.00	2
G	1.00	1.00	1.00	3
H	1.00	1.00	1.00	2
I	1.00	1.00	1.00	2
J	1.00	1.00	1.00	4
K	1.00	0.67	0.80	3
M	1.00	1.00	1.00	2
N	0.50	1.00	0.67	1
O	1.00	1.00	1.00	3
P	1.00	1.00	1.00	1
Q	1.00	0.50	0.67	2
R	0.33	0.50	0.40	2
S	1.00	0.67	0.80	3
T	0.80	0.80	0.80	5
U	0.50	1.00	0.67	2
V	0.67	0.50	0.57	4
W	1.00	0.50	0.67	2
X	1.00	1.00	1.00	4
Y	1.00	1.00	1.00	2
Z	0.80	1.00	0.89	4
Weighted avg	0.89	0.85	0.85	61

Table 3. Classification report on test set per letter

Sign Language Alphabet Recognition

Letter	Light	Angle	Dist.	Success	Correct	Incorrect	Other letters
C	Good	90°	30 cm	48	47	3	Q
C	Good	120°	50 cm	43	41	9	B, W
C	Good	145°	50 cm	23	23	27	/
C	Good	90°	50 cm	50	50	0	/
C	Good	90°	80 cm	48	48	2	/
C	Poor	90°	50 cm	45	40	10	B, R
M	Good	90°	30 cm	47	28	22	A, E, F, N
O	Good	90°	30 cm	50	14	36	C, Q, R
A	Good	90°	30 cm	50	42	8	E, T
L	Good	90°	30 cm	50	38	12	D

Table 4. Results of real-world testing for 50 frames across various lighting conditions, camera angles, distances, and letters

6.2. Figures

Figure 1 shows an example of the application in use, displaying a correct real-time prediction of the letter Y.

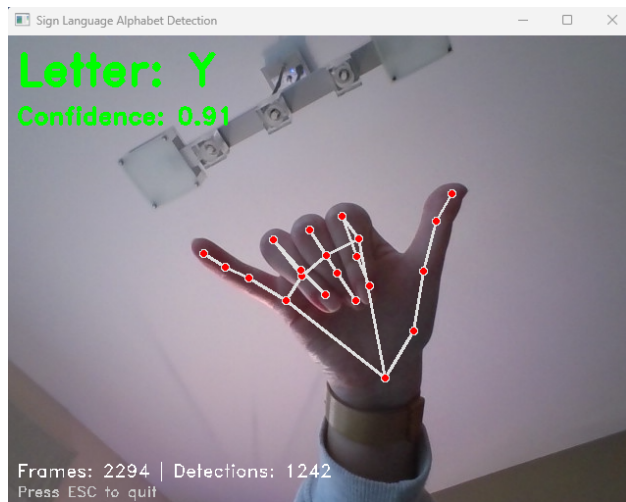


Figure 1. Example of the app while predicting the letter Y

References

- Google AI. Mediapipe hands: On-device real-time hand tracking. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2023. Accessed: 2025-11-14.
- Lugaresi, C., Tang, J., Nash, H., et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019. URL <https://arxiv.org/abs/1906.08172>.
- Pugeault, N. and Bowden, R. Spelling it out: Real-time asl fingerspelling recognition. In *2011 IEEE International Conference on Computer Vision Workshops*, pp. 1114–1119, 2011. URL <https://ieeexplore.ieee.org/document/6130325>.

Simon, T., Joo, H., Matthews, I., and Sheikh, Y. Hand keypoint detection in single images using multiview bootstrapping. *arXiv preprint arXiv:1704.07809*, 2017. URL <https://arxiv.org/abs/1704.07809>.

Tran, D., Bourdev, L., Fergus, R., Torresani, L., and Paluri, M. Learning spatiotemporal features with 3d convolutional networks. *arXiv preprint arXiv:1412.0767*, 2015. URL <https://arxiv.org/abs/1412.0767>.